

CYNA DEV

# Guide d'Utilisation

*CYNA DEV — un guide par technologie, avec les commandes*

**Projet :** CYNA DEV — Plateforme e-commerce SaaS de cybersécurité

**Groupe :** Charlotte Moerman · Nathan Noelijaona · Karl Negrevergne · Ylan Pernot

**Version :** 1.0

## 0. Vue d'ensemble rapide

---

Ce guide couvre, technologie par technologie, comment cloner, lancer, utiliser et déboguer le projet. Toutes les commandes ont été vérifiées en conditions réelles.

| Techno                      | Commande principale                     | URL / accès                                      |
|-----------------------------|-----------------------------------------|--------------------------------------------------|
| Git / GitHub                | git clone ...                           | github.com/PernotYlan/DEV-Projet-Site_E-commerce |
| Docker (tout en un)         | docker compose up --build               | http://localhost (site) · :9464 (API)            |
| Backend seul                | npm run dev                             | http://localhost:3000/api                        |
| Frontend seul               | npm run dev                             | http://localhost:5173                            |
| Base de données             | psql -h localhost -U app_user -d app_db | localhost:5432                                   |
| Back-office admin           | (via le site)                           | http://localhost/admin                           |
| Paieement — relais webhooks | stripe listen ...                       | —                                                |
| Chatbot — Ollama            | ollama serve                            | http://localhost:11434                           |
| Monitoring                  | docker compose up (dossier monitoring/) | Grafana :3000 · Prometheus :9090                 |

# 1. Git / GitHub

Le code est hébergé sur GitHub : PernotYlan/DEV-Projet-Site\_E-commerce.

## 1.1 Cloner le projet

```
git clone https://github.com/PernotYlan/DEV-Projet-Site_E-commerce.git
cd DEV-Projet-Site_E-commerce
```

## 1.2 Travailler sur une nouvelle fonctionnalité

Le projet suit une stratégie de branches par fonctionnalité (ex. la branche back-office a été fusionnée dans main via une Pull Request). Convention à respecter :

```
git checkout main
git pull
git checkout -b feature/nom-de-la-fonctionnalite

# ... modifications ...

git add .
git commit -m "feat: description claire du changement"
git push -u origin feature/nom-de-la-fonctionnalite
```

Puis ouvrir une Pull Request sur GitHub vers main pour revue de code avant fusion.

## 1.3 Récupérer les dernières modifications

```
git checkout main
git pull
```

## 1.4 Commandes utiles

| Commande                  | Effet                                                       |
|---------------------------|-------------------------------------------------------------|
| git status                | Voir les fichiers modifiés                                  |
| git log --online -10      | Voir les 10 derniers commits                                |
| git diff                  | Voir les changements non commités                           |
| git stash / git stash pop | Mettre de côté des changements en cours, puis les récupérer |
| git branch -a             | Lister toutes les branches (locales + distantes)            |

## 2. Docker & Docker Compose

C'est la méthode recommandée : un seul lancement démarre la base de données, le backend, le frontend, le chatbot (Ollama) et le relais des paiements Stripe.

### 2.1 Services orchestrés (docker-compose.yml, dossier code/)

| Service     | Rôle                                                          | Port exposé |
|-------------|---------------------------------------------------------------|-------------|
| db          | PostgreSQL 16, schéma + données initialisées automatiquement  | 5432        |
| backend     | API Node.js/Express                                           | 9464 → 3000 |
| frontend    | Site React buildé, servi par Nginx (+ proxy /api)             | 80          |
| ollama      | Serveur LLM local pour le chatbot                             | 11434       |
| ollama-init | Télécharge automatiquement le modèle llama3.2:1b au démarrage | —           |
| stripe-cli  | Relaie automatiquement les webhooks Stripe vers le backend    | —           |

### 2.2 Préparer l'environnement (une seule fois)

Le réseau `monitoring_net` est déclaré comme externe dans `docker-compose.yml` mais n'est créé par aucun des deux fichiers Compose : il faut le créer manuellement avant le premier lancement (sinon `docker compose up` échoue avec « `network monitoring_net declared as external, but could not be found` »).

```
docker network create monitoring_net
```

Puis copier les fichiers d'environnement :

```
cd code
cp backend/.env.example backend/.env      # renseigner ensuite les vraies valeurs
cp frontend/.env.example frontend/.env
```

### 2.3 Démarrer toute la stack

```
cd code
docker compose up --build

# En arrière-plan (détaché) :
docker compose up --build -d
```

Premier lancement : compter quelques minutes (build des images + téléchargement du modèle Ollama ~1,3 Go).

**Site :** <http://localhost>

**API :** <http://localhost:9464/api> (ou via le proxy : <http://localhost/api>)

### 2.4 Commandes utiles au quotidien

| Commande                                    | Effet                                |
|---------------------------------------------|--------------------------------------|
| <code>docker compose ps</code>              | Voir l'état des services             |
| <code>docker compose logs -f backend</code> | Suivre les logs du backend en direct |

| Commande                                                       | Effet                                                                |
|----------------------------------------------------------------|----------------------------------------------------------------------|
| <code>docker compose logs -f</code>                            | Suivre les logs de tous les services                                 |
| <code>docker compose restart backend</code>                    | Redémarrer un seul service                                           |
| <code>docker compose down</code>                               | Arrêter et supprimer les conteneurs (garde les volumes/données)      |
| <code>docker compose down -v</code>                            | Arrêter et supprimer AUSSI les volumes (⚠ efface la base de données) |
| <code>docker compose up --build backend</code>                 | Reconstruire l'image d'un seul service après modification du code    |
| <code>docker compose exec backend sh</code>                    | Ouvrir un terminal dans le conteneur backend                         |
| <code>docker compose exec db psql -U app_user -d app_db</code> | Ouvrir psql directement dans le conteneur de base de données         |

## 2.5 Monitoring (stack séparée, dossier code/monitoring/)

```
cd code/monitoring
docker compose up -d
```

**Grafana** : <http://localhost:3000> (identifiant : admin / mot de passe défini par GF\_SECURITY\_ADMIN\_PASSWORD, changeme par défaut – à changer)

**Prometheus** : <http://localhost:9090>

**Loki (logs)** : <http://localhost:3100> – consulté depuis Grafana, pas directement

*Penser à changer le mot de passe Grafana par défaut (GF\_SECURITY\_ADMIN\_PASSWORD dans monitoring/docker-compose.yml) avant tout déploiement réel.*

## 3. Backend (Node.js / Express) — sans Docker

Utile pour développer/déboguer le backend seul, avec rechargement automatique.

### 3.1 Windows — méthode rapide

Double-cliquer sur backend/DEMARRER-BACKEND.cmd. Installe les dépendances si besoin, puis lance l'API sur <http://localhost:3000>.

### 3.2 Terminal (Windows/Mac/Linux)

```
cd code/backend
cp .env.example .env      # renseigner DATABASE_URL, JWT_SECRET, etc.
npm install
npm run dev                # démarre avec nodemon (rechargement auto)

# Ou en mode production (sans rechargement) :
npm start
```

API : <http://localhost:3000/api>

### 3.3 Scripts utilitaires (backend/scripts/)

| Script                   | Commande                                            | Usage                                                                                      |
|--------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------|
| definir-mdp-admin.js     | node scripts/definir-mdp-admin.js<br>"MonMdp1!"     | Définit le mot de passe du compte admin par défaut (admin@cyna-it.fr)                      |
| test-email.js            | node scripts/test-email.js                          | Teste l'envoi d'un e-mail via la configuration SMTP actuelle                               |
| test-paiement.js         | node scripts/test-paiement.js                       | Simule un parcours de paiement complet (login → adresse → commande → paiement Stripe test) |
| start-stripe-listener.sh | (lancé automatiquement par le conteneur stripe-cli) | Relaie les webhooks Stripe vers le backend                                                 |

### 3.4 Variables d'environnement (backend/.env)

| Variable                                     | Rôle                                                          |
|----------------------------------------------|---------------------------------------------------------------|
| DATABASE_URL                                 | Chaîne de connexion PostgreSQL                                |
| JWT_SECRET / JWT_REFRESH_SECRET              | Secrets de signature des tokens (obligatoires au démarrage)   |
| STRIPE_SECRET_KEY /<br>STRIPE_WEBHOOK_SECRET | Clés Stripe (mode test conseillé en développement)            |
| SMTP_HOST / PORT / USER / PASS               | Serveur d'envoi d'e-mails                                     |
| OLLAMA_HOST / OLLAMA_MODEL                   | Adresse du serveur LLM et modèle utilisé par le chatbot       |
| FRONTEND_URL                                 | URL du site, utilisée pour CORS et les liens dans les e-mails |

## 4. Frontend (React / Vite) — sans Docker

---

### 4.1 Windows — méthode rapide

Double-cliquer sur frontend/DEMARRER-FRONTEND.cmd (le backend doit déjà tourner). Ouvre automatiquement le navigateur.

### 4.2 Terminal

```
cd code/frontend
cp .env.example .env
npm install
npm run dev          # serveur de développement Vite

# Build de production :
npm run build        # génère frontend/dist
npm run preview      # prévisualise le build de production
```

Site (dev) : <http://localhost:5173>

### 4.3 Mode démonstration sans backend

Le frontend peut fonctionner avec des données de démonstration intégrées, sans backend ni base de données — pratique pour une démo rapide ou en cas de panne du backend :

```
# Dans frontend/.env :
VITE MOCK=1
```

*Remettre VITE MOCK=0 pour retrouver le vrai comportement connecté à l'API.*

## 5. Base de données (PostgreSQL)

---

### 5.1 Se connecter

```
# Via Docker :  
docker compose exec db psql -U app_user -d app_db  
  
# En local (PostgreSQL installé sur la machine) :  
psql -h localhost -U app_user -d app_db
```

### 5.2 Réinitialiser le schéma depuis zéro

Avec Docker, les scripts d'initialisation (Documents/sqlcyna.sql, puis les scripts de seed/migration dans backend/scripts/) ne s'exécutent automatiquement qu'au premier démarrage du volume db\_data. Pour repartir de zéro :

```
docker compose down -v      # supprime le volume de données  
docker compose up --build   # les scripts d'init se rejouent automatiquement
```

### 5.3 Rejouer les scripts manuellement (sans Docker)

```
createdb app_db  
psql -h localhost -U app_user -d app_db -f Documents/sqlcyna.sql  
psql -h localhost -U app_user -d app_db -f code/backend/scripts/seed-catalogue.sql  
psql -h localhost -U app_user -d app_db -f code/backend/scripts/migration-stripe-customer.sql  
psql -h localhost -U app_user -d app_db -f code/backend/scripts/migration-adresses-telephone.sql
```

### 5.4 Requêtes utiles pour déboguer

```
-- Lister les utilisateurs  
SELECT id, email, role, email_verifie FROM Utilisateurs;  
  
-- Voir le catalogue  
SELECT id, nom, categorie_id, is_active FROM Produits;  
  
-- Voir les dernières commandes  
SELECT id, utilisateur_id, statut, cree_le FROM Commandes ORDER BY cree_le DESC LIMIT 10;
```



## 6. Le site — utilisation cliente

### 6.1 Parcours principal

| Étape | URL                       | Action                                        |
|-------|---------------------------|-----------------------------------------------|
| 1     | /                         | Accueil — carrousel, catégories, top produits |
| 2     | /catalogue/:categoryid    | Parcourir une catégorie (SOC / EDR / XDR)     |
| 3     | /produit/:id              | Détail d'une offre, ajout au panier           |
| 4     | /recherche                | Recherche avancée avec filtres                |
| 5     | /panier                   | Panier (accessible sans compte)               |
| 6     | /checkout                 | Tunnel d'achat (connexion requise)            |
| 7     | /confirmation/:commandeid | Récapitulatif de commande                     |

### 6.2 Créer un compte de test

```
curl -X POST http://localhost:3000/api/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"nom":"Test","prenom":"Utilisateur","email":"test@example.com","mot_de_passe":"MotDePasseFort123!"}'
```

Un e-mail de confirmation est envoyé (nécessite une config SMTP valide) ; sans SMTP configuré, le lien de confirmation reste visible dans les logs du backend en développement selon la configuration retenue.

### 6.3 Espace « Mon compte »

| Page               | URL                 |
|--------------------|---------------------|
| Profil             | /compte/profil      |
| Adresses           | /compte/adresses    |
| Moyens de paiement | /compte/paiements   |
| Abonnements        | /compte/abonnements |
| Commandes          | /compte/commandes   |

## 7. Back-office administrateur

### 7.1 Accès

**URL :** <http://localhost/admin> (ou <http://localhost:5173/admin> en dev)

Réservé aux comptes avec le rôle ADMIN. Un compte administrateur est pré-crée en base (admin@cyna-it.fr) ; définir son mot de passe avant la première connexion :

```
cd code/backend
node scripts/definir-mdp-admin.js "MonMotDePasseAdmin1!"
```

### 7.2 Fonctions disponibles

| Page                  | URL                                  | Fonction                                                               |
|-----------------------|--------------------------------------|------------------------------------------------------------------------|
| Tableau de bord       | /admin                               | KPIs, histogramme ventes/jour, paniers moyens par catégorie, camembert |
| Produits              | /admin/produits                      | Liste triable/paginée, sélection multiple                              |
| Nouveau produit       | /admin/produits/nouveau              | Création                                                               |
| Détail / modification | /admin/produits/:id                  | Consultation et édition                                                |
| Catégories            | /admin/categories                    | Gestion des catégories                                                 |
| Carrousel & accueil   | /admin/carrousel ·<br>/admin/accueil | Personnalisation de la page d'accueil                                  |
| Commandes             | /admin/commandes                     | Supervision des commandes                                              |
| Abonnements           | /admin/abonnements                   | Supervision des abonnements                                            |
| Utilisateurs          | /admin/utilisateurs                  | Gestion des comptes clients                                            |
| Messages              | /admin/messages                      | Messages du formulaire de contact                                      |

## 8. Paiement — Stripe (tests)

---

### 8.1 Avec Docker (automatique)

Le conteneur stripe-cli relaie déjà automatiquement les webhooks vers le backend. Rien à faire de plus que renseigner STRIPE\_SECRET\_KEY dans .env.

### 8.2 Sans Docker — Windows

Double-cliquer sur backend/webhook-stripe.cmd et laisser la fenêtre ouverte pendant les tests de paiement.

### 8.3 Sans Docker — Mac/Linux

```
# Installer Stripe CLI : https://stripe.com/docs/stripe-cli  
stripe listen --forward-to localhost:3000/api/paiement/webhook
```

### 8.4 Cartes de test Stripe

| Numéro de carte     | Comportement                       |
|---------------------|------------------------------------|
| 4242 4242 4242 4242 | Paiement accepté                   |
| 4000 0000 0000 0002 | Carte refusée                      |
| 4000 0025 0000 3155 | Authentification 3D Secure requise |

Date d'expiration : n'importe quelle date future. CVC : n'importe quel code à 3 chiffres.

## 9. Chatbot (Ollama)

---

Le chatbot s'appuie sur un LLM auto-hébergé (llama3.2:1b) via Ollama, pas sur une API tierce payante.

### 9.1 Avec Docker (automatique)

Le service ollama démarre automatiquement, et ollama-init télécharge le modèle au premier lancement (~1,3 Go, peut prendre plusieurs minutes).

### 9.2 Sans Docker

```
# Installer Ollama : https://ollama.com/download
ollama serve
ollama pull llama3.2:1b

# Dans backend/.env :
# OLLAMA_HOST=http://localhost:11434
# OLLAMA_MODEL=llama3.2:1b
```

### 9.3 Vérifier que le chatbot répond

```
curl -X POST http://localhost:3000/api/chatbot/chat \
-H "Content-Type: application/json" \
-d '{"message": "Bonjour, quels sont vos tarifs ?"}'
```

## 10. Aide-mémoire — toutes les commandes

---

| Objectif                                  | Commande                                                                                                                                             |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cloner le projet                          | git clone<br><a href="https://github.com/PernotYlan/DEV-Projet-Site_E-commerce.git">https://github.com/PernotYlan/DEV-Projet-Site_E-commerce.git</a> |
| Créer le réseau Docker manquant           | docker network create monitoring_net                                                                                                                 |
| Tout démarrer (Docker)                    | cd code && docker compose up --build                                                                                                                 |
| Tout arrêter                              | docker compose down                                                                                                                                  |
| Tout arrêter + effacer les données        | docker compose down -v                                                                                                                               |
| Voir les logs du backend                  | docker compose logs -f backend                                                                                                                       |
| Backend seul (dev)                        | cd code/backend && npm run dev                                                                                                                       |
| Frontend seul (dev)                       | cd code/frontend && npm run dev                                                                                                                      |
| Se connecter à la base                    | docker compose exec db psql -U app_user -d app_db                                                                                                    |
| Définir le mot de passe admin             | node scripts/definir-mdp-admin.js "MotDePasse1!"                                                                                                     |
| Relayer les webhooks Stripe (hors Docker) | stripe listen --forward-to localhost:3000/api/paiement/webhook                                                                                       |
| Démarrer le monitoring                    | cd code/monitoring && docker compose up -d                                                                                                           |
| Démarrer Ollama (hors Docker)             | ollama serve && ollama pull llama3.2:1b                                                                                                              |